

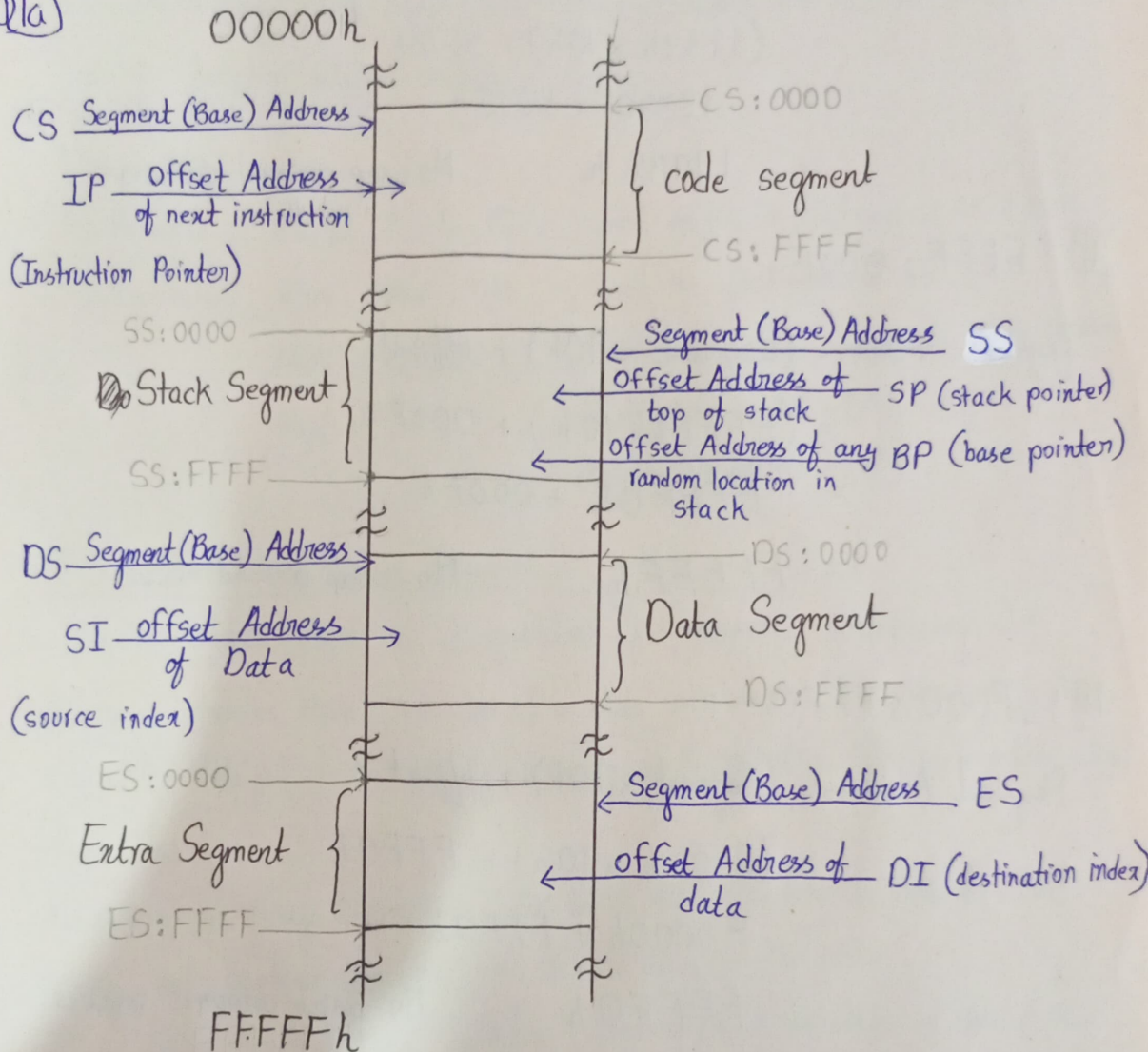
COAL-Assignment 1

Name: Sana Munir Alam

Roll#: 24K-0573

Section: BSCS-3E

Q1a)



To calculate a physical address from a segment: offset pair. Use this formula:

$$\text{Physical Address} = (\text{Segment} * 10h) + \text{offset}$$

eg: 22A0:1243h } segment: offset

$$\begin{aligned}\text{Physical Address} &= (22A0h * 10h) + 1243h \\ &= 22A00h + 1243h \\ &= 23C43h\end{aligned}$$

Q1bi) 1234 : 5678

$$\begin{aligned}\text{Physical Address} &= (\text{Segment} \times 10h) + \text{Offset} \\ &= (1234h \times 10h) + 5678h \\ &= 12340h + 5678h \\ &= 179B8h \quad \text{No wrap around occurs}\end{aligned}$$

ii) FFFF : 000F

$$\begin{aligned}\text{Physical Address} &= (\text{Segment} \times 10h) + \text{offset} \\ &= (FFFFh \times 10h) + 000Fh \\ &= FFFF0h + 000Fh \\ &= FFFFFh \quad \text{No-wrap around occurs}\end{aligned}$$

iii) F000 : FF0

$$\begin{aligned}\text{Physical Address} &= (\text{Segment} \times 10h) + \text{offset} \\ &= (F000h \times 10h) + FF0h \\ &= F0000h + FF0h \\ &= FFFF0h \quad \text{No-wrap around occurs}\end{aligned}$$

iv) F800 : 9000

$$\begin{aligned}\text{Physical Address} &= (\text{Segment} \times 10h) + \text{offset} \\ &= (F800h \times 10h) + 9000h \\ &= F8000h + 9000h \\ &= 101000h\end{aligned}$$

as $101000h > FFFFFh$ therefore wrap around will occur
 $\therefore$ Physical Address after wrap around is $01000h$.

Q2) EAX (Extended Accumulator Register):

It is an accumulator, the preferred register for performing arithmetic operations (eg: ADD, SUB, DIV, MUL), control instructions and storing function return values.

Use Case:

- Arithmetic:- `ADD EAX, 10` ; Add 10 to whatever is in EAX
- Multiplication: `MOV EAX, 200` ; Load the first operand into EAX
`MOV ECX, 10` ; Load the second operand in other register
`MUL ECX` ; Instruction: $EAX = EAX * ECX$
Result stored in EAX

→ EBX (Extended Base Register)

It is often used as a pointer for memory addressing. It holds a stable, base address of a data structure (like an array or object). Helpful in addressing memory with offsets. (Indirect addressing)

Use Case

Accessing an Array Element:- EBX holds the starting address of the array, and another register (like ESI) acts as the index

`MOV ebx, OFFSET myArray` ; EBX points to start of my Array
`MOV esi, 4` ; ESI is the index

`MOV eax, [ebx + esi*4]` ; Indexed addressing = $EAX = myArray[4]$

→ ECX (Extended Counter Register)

It is used as a primary loop counter. Also used in shift/rotate instructions to specify how many times to shift. Instructions like LOOP and REP automatically use and decrement ECX to control repetition.

Use Case:

Controlling a Loop: ECX is initialised to the number of iterations

MOV ecx, 10 ; We want to loop 10 times

ArrayLoop : ; Start of loop

; any code here which we want to do 10 times

LOOP ArrayLoop ; Decrements ECX, jumps to ArrayLoop if ECX is not equal to 0.

→ EDX (Extended Data Register)

It often acts as a secondary companion to EAX in arithmetic operations (like Multiplication, division of large numbers)

eg the upper 32 bits of a 64-bit result from multiplication or the remainder in division.

Use Case :

- Division :-
MOV eax, 50 ; Divident (Lower half)
MOV edx, 0 ; clear EDX for the upper half
MOV ecx, 4 ; Divisor
div ecx ; Instruction ; EAX = quotient (5), EDX = remainder (5)
- Multiplication :-
MOV eax, 0FFFFFFFFh
MOV ecx, 0FFFFFFFFh
IMUL ecx ; EDX : EAX = EAX * ECX

The upper 32 bit in EDX while lower are in EAX

→ ESI and EDI (Extended Source Index and Extended Data Index)

The two are used by high-speed memory transfer instruction, like memory copying and string operation.

ESI : Used as a source pointer in string/memory operation

EDI : Used as a destination pointer in string/memory operation

Use Case :

- Copying a Block of Memory

MOV esi, ^{OFFSET} Source Buffer ; ESI points to where data comes from
MOV edi, OFFSET dest Buffer ; EDI points to where data goes TO
MOV ecx, LENGTHOF source Buffer ; ECX = no. of bytes to copy
rep movsb ; Repeat : copy byte from [ESI] to [EDI],
increment both pointer and decrement ECX until 0.

→ ESP (Extended Stack Pointer)

It always points to the top of the stack in memory. The stack is a LIFO (Last in First out) data structure crucial for function calls, local variables, and saving register states.

It is managed automatically by :

PUSH instruction : decrements ESP and then stores data

POP instruction : retrieves data and then increments ESP

Use Case :

We can use function call "push" to return addresses onto the stack via ESP.

eg :- push eax ; ESP decrease, value stored in stack

→ ~~EBP~~ (Extended Base Pointer) also known as Extended Frame Pointer :-

It is used by High Level Language to reference function parameters and local variables on the stack. It acts as a base pointer for the current function's stack frame. While ESP moves as items are pushed/popped, a function sets up EBP to point to a fixed location on the stack. This creates a consistent reference point (eg., [EBP-4], [EBP+8]) to access function parameters and local variables.

Use Case : Used in a function call either to access function parameter or local variable

~~MOV eax, Ebp~~

push ebp	; save the old frame pointer
mov ebp, esp	; set EBP to point to the top of stack
mov eax, [ebp+8]	; access function parameter
mov ebx, [ebp-4]	; access local variable
pop ebp	; restore the old frame pointer
ret	; return

→ EIP (Extended Instruction Pointer)

It is a very important register as it contains the memory address of the next instruction to be fetched and executed by the CPU. It automatically increments after each instruction. It is changed by control flow instructions (JMP, CALL, LOOP, JNE, JE). This directly load a new address into EIP

Use Case:

It is typically CPU cycling through "fetch the instruction at [EIP], execute it, update EIP."

However we can use jump/calls to change EIP

eg:- jmp Label ; EIP is set to address of 'label'

Q3) Little Endian System:-

Least significant bit is stored first

0x1234ABCD
 ↓ ↓
 most least
 significant significant

Memory Address Stored Value

4000h	CD
4001h	AB
4002h	34
4003h	12

Big Endian System:

Most significant byte is stored first

0x 1234ABCD
 ↙ ↘
 most least
 significant significant

memory Address	Stored value
4000h	12
4001h	34
4002h	AB
4003h	CD

Q4) MOV AX, 8F7AH

} Instruction 1

Sol:

AX = 8F7Ah

CF = unchanged = 0

SF = unchanged = 0

Assuming CF and SF were initially 0

ADD AX, 7AF8H

} Instruction 2

Sol:

AX = 8F7Ah + 7AF8h

AX = 0A72h

CF = 1

SF = 0

$$\begin{array}{r} \overset{1}{8}\overset{1}{F}\overset{1}{7}A \\ + 7AF8 \\ \hline 1\ 0A72 \\ \text{carry bit} \end{array}$$

MOV BX, 0FA77h

} Instruction 3

Sol:

BX = FA77h

CF = 1 unchanged

SF = 0 unchanged

INC BX

} Instruction 4

Sol:

BX = FA77h + 1

BX = FA78h

CF = 1 (unchanged)

SF = 1 as FA78h where F is 1111 $\therefore$ MSB is 1
indicating negative number in signed arithmetic

Q5i)

Seconds	EQU	60	; 60 seconds
Minutes	EQU	60	; 60 seconds in 1 minute
Hours	EQU	24	; 24 hrs in one day
Days	EQU	7	; 7 days in one week

ii) To calculate no. of seconds in a week

No. of seconds in week = Seconds * Minutes * Hours * Days



• data

Seconds	EQU	60
Minutes	EQU	60
Hours	EQU	24 24
Days	EQU	7

• code

main PROC

; INSTRUCTION FLOW

mov EAX, Seconds ; EAX = 60

imul EAX, Minutes ; EAX = EAX * 60 $\Rightarrow$ 60 * 60

imul EAX, Hours ; EAX = EAX * 24 $\Rightarrow$ 3600 * 24

imul EAX, Days ; EAX = EAX * 7 $\Rightarrow$ 86400 * 7

; The final result of 604800 seconds in a week is stored
; in EAX register

main ENDP

END Main

(iii) SOLVED on Visual Studio, screen shot attached below

Q7a) Little Endian System:

32-bit value: $0x A1B2C3D4$

most significant Byte least significant Byte

Starting memory: $3000h$

Memory Address	Stored Value	
$3000h$	D4	LSB
$3001h$	C3	
$3002h$	B2	
$3003h$	A1	MSB

7b) If the system is big-endian the memory layout will flip.

Now the most significant byte is stored at the lowest memory. Subsequent bytes are stored in decreasing order of significance.

32-bit value: $0x A1B2C3D4$

most significant byte least significant byte

Starting memory: $3000h$

Memory Address	Stored Value	
$3000h$	A1	MSB
$3001h$	B2	
$3002h$	C3	
$3003h$	D4	LSB

7c) code

`MOV EAX, [3000h]`

as x86 follow little endian it will load the 4 bytes correctly into the 32 bit register

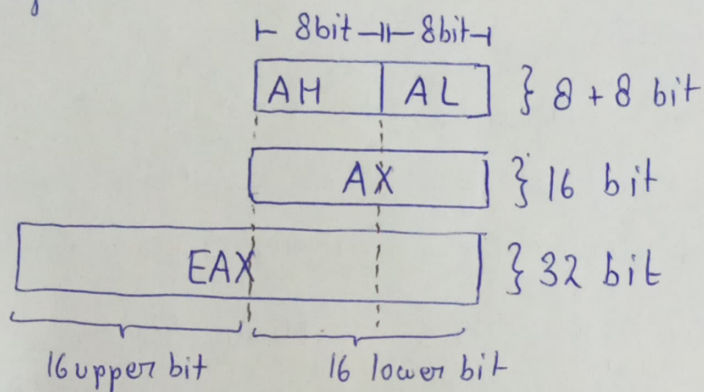
result $\Rightarrow$ $EAX = 0x A1B2C3D4$

Q8) The 32-bit general purpose registers can be split into 16-bit registers and further into 8-bit parts because of backward compatibility and efficient data handling.

Initially x86 structure was started with 16-bit processor who used AX, BX, CX and DX register. So, when 32-bit processor

were made, they needed to extend the 16-bit register to 32-bit while ensuring that the new software could still run old 16-bit code.

Hence 16-bits AX register became the lower 16-bits of the new 32-bit EAX register. This design creates a logical hierarchy within the CPU, where the smaller registers are part of the larger register.



Situation where using a smaller register is more efficient than a full 32-bit register is for example processing ASCII characters. They occupy 1 byte each so it's efficient using a smaller register unlike 32-bit one where 3 byte space would be wasted.

Hence using smaller register saves instructions, memory, and can be faster as we avoid dealing with unused bits.